# Ders 7 Çalışma Özeti

## Ders 7 Çalışma Özeti

### Genel Konular

- Vize öncesi duyurular: sınav tarihi, sınav türü ve gereksinim belgesi teslimi.
- Yazılım ölçümü (software metrics) tekrarı: McCabe, Halstead, büyüklük ve güvenirlik ölçüleri.
- Ölçme (measuring), ölçüt (metric) ve ölçüm (measurement) kavramları.
- İç ve dış kalite ölçütleri (internal / external quality).
- Ölçme süreci: tanımlama, toplama, hesaplama, yorumlama, geri besleme.
- CK metrikleri: nesneye yönelik yazılımlar için sınıf ölçütleri.
- Yazılım test planlama ve yönetimi tekrarı: test spesifikasyonu, test planı, test senaryosu, test ekibi.
- Optimum test miktarı ve bakım (maintenance) aşaması.
- Dönem başından itibaren genel tekrar: McCall kalite modeli, kalite aktiviteleri, V model, gözden geçirme ve inceleme.
- Test süreci adımları: birim, bütünlük, regresyon, sistem ve onaylama testleri.
- Test tipleri: saydam kutu (white box), kara kutu (black box), gri kutu (gray box).
- Hata giderme (debugging) ve hata düzeyleri.

### Hocanın Özellikle Vurguladığı Kısımlar

- Vize, bir sonraki hafta (vize haftası) bölümün ara sınav programındaki saatte yapılacak. Öğrencilere göre saat 13:00-15:00 aralığında. Sınav bir saati geçmeyecek.
- Sınavın test mi klasik mi olacağı henüz belli değil. Sınav sistemi hakkında gelecek bilgiye göre karar verilecek.
- Soru tipleri: derste görülenlerle yorum sorusu, kavram kıyaslaması, belki küçük problem çözümleri.
- Konuları ezber gibi değil, birbiriyle kıyaslayarak ve mantık sorularına cevap verecek şekilde çalışın.
- Hoca, tekrar slaytlarının özetlenmiş hâlini ders sonrası paylaşacak. Tekrar slaytları temel konuları verir; ayrıntılı ders slaytlarını da gözden kaçırmayın.
- Gereksinim belgesi (SRS) için son gün bu derstir (17 Kasım, gün sonuna kadar). Bu tarihten sonra gönderenlere daha düşük baremden not verilecek.
- “Ölçülemeyen yönetilemez” (Tom DeMarco). Kaliteyi iyileştirmek için önce var olan durumu ölçmek gerekir.
- Yazılım geliştiriciler birim testini yapmakla yükümlüdür. Diğer test adımlarını bağımsız bir test grubu yapmalıdır.
- İyi bir testçi mükemmele ulaşamayacağını, iyinin yeterli olacağını bilmelidir.
- Slayttaki eski V model şeklinde doğrulama ve geçerleme yer değiştirmiş. Doğru kullanım: validation = geçerleme, verification = doğrulama.
- Ölümcül hata varsa yalnız teslim değil, test de durdurulur.
- Kalite özelliği “doğruluk” (correctness) spesifikasyona uygunluktur. Bu yüzden projede önce gereksinim spesifikasyonu yazdırıldı. Seçilen ürünün bu spesifikasyona uyup uymadığı kontrol edilecek.

### Kısa Tekrar Notları

- Ölçme: bir varlığın özelliğini sayısal veya derecelendirilmiş veri olarak ifade etmek.

- Ölçüt: ölçülecek özelliğin ölçme şekli. Örnek: Cyclomatic Complexity, LCOM, LOC, function point.
- Ölçüm: belli bir ölçüte göre yapılan ölçme eyleminin sonucu.
- Ölçme adımları: tanımlama, toplama, hesaplama (analiz), yorumlama, geri besleme.
- McCabe ölçüsü (döngüsel karmaşıklık) kontrol deyimlerinden doğan temel yol sayısını verir. Değer 10'dan büyükse modülün bakım yapılabilirliği azalır.
- Büyüklük ölçüleri: satır sayısı (LOC), bin satır (KLOC), function point. Satır sayısı programlama diline bağlıdır.
- Güvenirlik: belli kullanım süresinde çıkan ve düzeltilen hata sayılarından istatistiksel olarak bulunur.
- Dış kalite ölçütleri kullanıcıyı ilgilendirir: kullanışlılık, performans, güvenlik, doğruluk, güvenirlik.
- İç kalite ölçütleri geliştiriciyi ilgilendirir: bakım kolaylığı, esneklik, taşınabilirlik, yeniden kullanılabilirlik, okunabilirlik, anlaşılabilirlik, test edilebilirlik.
- CK metrikleri: ağırlıklı metot sayısı, kalıtım ağacı derinliği, alt sınıf sayısı, yanıt kümesi, bağlaşım (coupling), kohezyon eksikliği (LCOM).
- Optimum test miktarı: kaçırılan hata sayısı eğrisi ile test maliyeti eğrisinin kesiştiği noktadır. Her projede farklıdır.
- Bakımda en çok yapılan iş uyarlamadır. İyileştirme ve hata düzeltme bunun yaklaşık yarısı kadardır.
- Saydam kutu testi ağırlıkla birim testte, kara kutu testi ağırlıkla kullanıcı testinde kullanılır. İkisi birbirini tamamlar.
- Hata düzeyleri: ölümcül, kritik, büyük, orta, küçük, görünüm.

## Detaylı Açıklamalar

### Sınav ve Proje Duyuruları

- Hoca önceki hafta vizenin 9. haftada yapılabileceğini söylemişti. Vize haftası olduğu için sınav diğer derslerle aynı programa göre yapılacak.
- Sınavdan önce genel tekrar yapılması öğrencilerden onay aldı. Dersin ikinci yarısı genel tekrara ayrıldı.
- Gereksinim belgesi teslimi önce 6. haftaydı. 7. haftaya uzatıldı. Bu dersin günü bitene kadar gönderenlerden puan kırılmayacak.
- Gereksinim raporundan ve sınavdan sonra projelerde test aşamasının nasıl yapılacağı ayrıca konuşulacak.
- Kalite iyileştirme konusu bu tekrarın dışında. Sınavdan sonraki hafta işlenecek.

### Yazılım Ölçümü Tekrarı

- Yazılım ölçümü yazılımı büyüklük, karmaşıklık ve iç yapı özelliklerine göre değerlendirir.
- Halstead metriği aritmetiksel ölçümler verir: program hacmi, program uzunluğu, aritmetik ifadelerin yoğunluğu gibi.
- Algoritma karmaşıklığı da yazılım hakkında sayısal değer verir.
- McCabe ölçüsünde graf kullanılır:
  - Düğümler program deyimleridir. Kritik olanlar kontrol deyimleridir (`if`, döngü kontrolü).
  - Kenarlar düğümleri bağlayan yollardır.
  - Döngüsel karmaşıklık kenar ve düğüm sayısından ya da graftaki kapalı alan sayısından bulunur.
  - Yaygın formül: $V(G) = E - N + 2$ ($E$: kenar, $N$: düğüm). Kapalı alan sayısı + 1 de aynı sonucu verir.
- Büyüklük ölçüsüyle şu tür yargılara varılır: belli satır sayısını geçen modülü alt modüllere ayırmak iyi olur.
- Satır sayısı dile bağlı olduğu için diller arasında çevrim yapılır. Örnek: bir alt seviye dildeki satır sayısının C'deki karşılığı.
- Güvenirlik modelinde sayılanlar:
  - belli kullanım süresinde çıkan hata sayısı,
  - düzeltilebilen hata sayısı,
  - düzeltmeden sonra kalan hata sayısı.

### Ölçme, Ölçüt ve Ölçüm

- Ölçme (measuring): somut veya soyut bir varlığın özelliğini sayısal ya da derecelendirilmiş veri olarak ifade etmek.
  - "Ara sınav çok zordu" demek derecelendirilmiş ama genel bir ölçüdür.
  - Hedef, yazılımla ilgili ölçüleri sayısal değere veya sayısal aralıklara getirmektir.
- Ölçüt (metric): ölçülecek özelliğin ölçme şekli.
  - Sıcaklık için santigrat veya Fahrenheit gibi.
  - Yazılımda: döngüsel karmaşıklık, LCOM, coupling, sınıf ağacı derinliği, LOC, KLOC, function point.
- Ölçüm (measurement): belli bir ölçüte göre yapılan ölçme eyleminin sonucu.
  - Function point için girdi, çıktı ve dosya sayıları sayılır. Bunlar doğrudan ölçümdür.
  - Function point bu sayılardan hesaplanır. Bu dolaylı ölçümdür.
- Neden ölçüyoruz?
  - Ne kadar iyi bir ürün çıkardığımızı bilmek için.
  - Bu büyüklükte bir yazılıma ne kadar zaman ve para harcadığımızı bilmek için.
  - Yeni projede büyüklük tahmini yapıp maliyet kestirmek için.
  - Kaliteyi artırmak için neyi değiştireceğimizi görmek için.

### Kalite Ölçütleri: Modeller

- ISO 9126 kalite ölçütlerini tek tek sayar: işlevsellik, uygunluk, doğruluk, güvenlik, güvenirlik, olgunluk vb.
- McCall aynı ölçütleri gruplara ayırır (ürün işletimi, ürün değişikliği, ürün geçişi).
- McConnell ölçütleri iç ve dış kalite ölçütleri olarak ikiye ayırır.
- Dış kalite ölçütleri: yazılıma dışarıdan bakan kişinin, yani kullanıcının istediği özellikler.
  - Kullanışlılık, performans, hızlı cevap süresi, veri kapasitesi.
  - Yetkisiz kullanımı önleyen güvenlik.
  - Doğru çalışma ve bu doğruluğun güvenilir olması (reliability).
- İç kalite ölçütleri: geliştiricinin kod veya sözde koda (pseudo code) bakarak ölçeceği değerler.
  - Bakım kolaylığı (maintainability): yetenek eklemenin, hata gidermenin ve başarımı artırmanın kolay olması.
  - Esneklik (flexibility): yazılımın tasarlandığı uygulama dışında çalışması için gereken değişikliğin az olması.
  - Taşınabilirlik (portability): farklı donanım ve işletim sistemlerine kolay aktarılabilmesi.
  - Yeniden kullanılabilirlik (reusability): sistemin bütününün veya parçalarının başka sistemlerde kullanılabilmesi.
  - Okunabilirlik (readability): kaynak kodun kolay incelenmesi. Dilin standartlarına uygun yazım ve yerinde açıklamalarla sağlanır.
  - Anlaşılabilirlik (understandability): sistem, bileşen ve kod düzeylerinde kolay anlaşılma. Okunabilirlik yalnız kod düzeyini kapsar.
  - Test edilebilirlik (testability): gereksinimlerin karşılanıp karşılanmadığının bileşen ve sistem düzeyinde test edilebilmesi.

### Ölçme Süreci

| Adım | Ne yapılır | Örnek |
|---|---|---|
| Tanımlama | Yazılıma uygun ölçütler seçilir | Yaklaşım (yapısal, nesneye yönelik) ve tür (gerçek zamanlı, gömülü) dikkate alınır |
| Toplama | Seçilen ölçütlerin gerektirdiği veri toplanır | Nesneye yönelik programda CK metriklerinin her biri için veri |
| Hesaplama (analiz) | Ham ölçülerden ölçütler hesaplanır | Girdi, çıktı, dosya sayılarından function point |

| Adım | Ne yapılır | Örnek |
|---|---|---|
| Yorumlama | Ölçüm değerlerinden anlam çıkarılır | “Bu function point değeri büyük yazılım sınıfına girer” |
| Geri besleme | Sonuçlar yazılım ekibine bildirilir | Ekip sonuçları kullanarak yazılımı iyileştirir |

- Hesaplamanın otomatik yapılması tercih edilir. Bunun için altyapı yazılımları hedeflenir.
- İyileştirme için önceki değerleri bilmek gerekir.

**CK Metrikleri**

- CK metrikleri yalnız nesneye yönelik yazılımlar için geçerlidir. Sınıflar üzerinden sayılır.
- Metrikler:
  - Sınıftaki ağırlıklı metot sayısı (WMC).
  - Kalıtım ağacının derinliği (DIT).
  - Alt sınıf sayısı (NOC).
  - Sınıflar arası bağlaşım (CBO).
  - Sınıfın yanıt kümesinin eleman sayısı (RFC).
  - Metotlardaki kohezyon eksikliği (LCOM).
- Bu ölçütlere eleştiriler de vardır. Her biri istenen değerin karşılığını tam vermez.
  - Örnek soru: metot karmaşıklığı neye göre belirlenecek?
  - Çok metotlu bir sınıfta sorumlulukların dağıtılması zor olur gibi yorumlar yapılabilir.

**Test Planlama ve Yönetimi Tekrarı**

- Test kapsamındaki işlemler: test metodolojisine karar verme, test planlama, test tasarımı, testin gerçekleştirilmesi.
  - Test metodolojisi, yazılım geliştirme metodolojisi seçilirken belirlenir.
  - Test verisi test tasarımı sırasında hazırlanır.
- Test spesifikasyonu testlerin tanımlanması, planlanması, düzenlenmesi ve belgelenmesi için oluşturulur.
- Spesifikasyonda her test için yazılanlar:
  - testin tanımı, amacı ve test edilen modüller,
  - kullanılan özel araçlar (tool),
  - gider ve zamanlama,
  - test verisi,
  - beklenen sonuçlar ve gerçek sonuçlar.
- Test adımları için Gantt diyagramı hazırlanır. Test için gider tahmini yapılır.
- Ortam ve kaynak örneği: birim test için kullanılan dile uygun bir birim test aracı seçilir.
- Test yönetimi geri beslemeli bir süreçtir. Hata mesajları sınıflandırılır ve belgelenir.
- IEEE’nin test dokümantasyonu standardı test spesifikasyonunun nasıl oluşturulacağını verir.
- Test spesifikasyonu da konfigürasyon yönetimine girer. Gereksinim, tasarım ve kod belgeleri gibi bir baseline ile izlenir.

**Test Planı ve Test Senaryosu İçeriği**

- Test planında bulunanlar:
  - test stratejisi,
  - test edilecek öğeler,
  - doğrulama yöntemleri,
  - testlerin tamamlanma kriterleri (ne zamana kadar hata aranacak),
  - hata ve test sonuç raporu biçimi,
  - test sorumlulukları,
  - test ortamı ve test araçları,
  - eleman eğitim ihtiyacı,
  - test takvimi (Gantt diyagramı),
  - test risk yönetimi,

  - testin beklenen çıktıları.
- Test senaryosunda bulunanlar:
  - adı, kimliği, yazarı, tarihi,
  - ilgili gereksinimler ve testin amacı,
  - ön koşullar ve varsayımlar,
  - test girdileri,
  - senaryo adımları,
  - beklenen sonuçlar.
- Senaryo örneği (öğrenci işleri sistemi): öğrencinin dönem içinde derse kaydolması, öğrencinin transkript istemesi.

**Test Ekibi ve Optimum Test Miktarı**

- Test ekibinde bağımsızlık önemlidir. Testi yapan grubun geliştirici grubunda olmaması tercih edilir.
- Test ekibinin görevi:
  - planda belirtilen testler için hazırlık yapmak,
  - testleri yürütmek,
  - bulunan hataları yazılımcıya bildirmek,
  - test raporlarını konfigürasyon yönetimi belgelerine eklemek.
- Optimum test miktarı:
  - Kaçırılan hata sayısını en aza indirmek istenir.
  - Test maliyetini de çok artırmamak istenir.
  - İki eğrinin kesişim noktası optimum test miktarıdır.
  - Test maliyetinin proje maliyet planında belli bir yeri vardır.

**Test Faaliyetlerinin Geliştirme Aşamalarına Dağılımı**

| Aşama | Test faaliyeti |
|---|---|
| Planlama | Kaba sistem test planı |
| Gereksinim analizi | Her işlevin nasıl test edileceği planlanır |
| Tasarım | Modül test planı; yapı diyagramları ve mimari bu plana yardımcı olur; test tanımları ve test eğitimi |
| Gerçekleme | Birim testleri, bütünlük testleri, parçalar için sistem testi |
| Kurulum | Kullanıcı testleri ve bütün üzerinde bazı sistem testleri |
| Teslim sonrası (bakım) | Kalitenin sürdürülmesi |

**Bakım Aşaması**

- Bakım: yazılımın korunması, onarılması ve geliştirilmesi.
- Bakımda yapılanlar: sonradan görülen hataların düzeltilmesi, iyileştirme, uyarlama, geliştirme.
- Bakımda en çok uyarlama yapılır. Bir çalışmaya göre iyileştirme ve hata düzeltme bunun yarısı kadardır.
- Geniş bir programın kullanımı sırasında testte bulunmamış hatalar ve bazı tasarım hataları ortaya çıkar.
- Bakım maliyetini azaltmak için:
  - standart programlama dilleri ve standart belgeler kullanılmalı,
  - test programlarından yararlanılmalı,
  - tasarımda gözden geçirme iyi yapılmalı.
- Bakım yapılabilirlik (maintainability): kolay test edilebilirlik, basitlik, değiştirilebilirlik, taşınabilirlik, güvenirlik ve esneklik.
- Yazılım konfigürasyon sistemi her baseline'daki özellikleri bir veri tabanı gibi saklar. Güncel hâlin tüm elemanlarını kullanıma hazır tutar.

**Genel Tekrar: Yazılım Mühendisliğinde Kalite**

- Yazılım mühendisliği katmanlı bir yapıdır. En altta kalite bakış açısı vardır. Üstünde süreçler, yöntemler ve araçlar gelir.
- Tanım: kaliteli yazılımı en ekonomik şekilde ve mühendislik yöntemleriyle elde etmeye yarayan bilim dalı.
- Kullanılan araç hem yönteme hem de süreç adımına uygun olmalıdır.
- McCall kalite üçgeni:

| Bakış açısı | Kalite faktörleri |
|---|---|
| Ürün işletimi (kullanıcı) | Doğruluk, güvenirlik, kullanışlılık, bütünlük, etkinlik |
| Ürün değişikliği | Bakım yapılabilirlik, esneklik, test edilebilirlik |
| Ürün geçişi | Taşınabilirlik, yeniden kullanılabilirlik, birlikte çalışabilirlik |

- Kalite özelliklerinin tanımları:
  - Doğruluk (correctness): spesifikasyonlara uygunluk.
  - Güvenirlik (reliability): işlevleri istenen duyarlılıkla yerine getirme.
  - Verimlilik (efficiency): bellek, işlem süresi ve kodlama gideri açısından kaynak kullanımı.
  - Bütünlük (integrity): yetkisiz kişilerin yazılıma erişememesi.
  - Kullanışlılık (usability): öğrenme, işletme, girdi hazırlama ve çıktı yorumlamanın kolaylığı.
  - Bakım kolaylığı: hatanın yerini bulma ve düzeltme kolaylığı.
  - Test edilebilirlik: doğruluğu test etmenin kolaylığı.
  - Taşınabilirlik: başka donanım veya yazılım ortamına düşük maliyetle taşınabilme.
  - Yeniden kullanılabilirlik: tümünün veya bir bölümünün başka uygulamada kullanılabilmesi.
  - Birlikte çalışabilirlik (interoperability): bir sistemin diğeriyle birlikte çalışabilmesi.
- Kalite faktörleri doğrudan ölçülmez. Metriklerin katsayılarla bir regresyon denklemine konmasıyla hesaplanır.
- Faktör ile metrik ilişkisine örnekler:
  - Birlikte çalışabilirlik için modülerlik, iletişim genelliği ve veri genelliği önemlidir.
  - Yeniden kullanılabilirlik için yazılım sistemi bağımsızlığı ve makine bağımsızlığı önemlidir.

**Kaliteli Yazılım İçin Aktiviteler ve Kalite Maliyeti**

- Aktiviteler:
  - standartlara uymak,
  - doğrulama ve geçerleme (verification, validation),
  - gözden geçirme ve denetim (review),
  - test,
  - hata ve kusur analizi,
  - değişim ve konfigürasyon yönetimi,
  - eğitim,
  - risk yönetimi (geliştirmede ve testte her aşamada).
- Bu aktivitelerin teslimden önce yapılması gerekir.
- Kalite maliyetleri:
  - Hata önleme maliyeti: kalite planlama, formal teknik gözden geçirme, test oluşturma, eğitim.
  - İç hata maliyeti: yazılım evinde, kullanıcıya verilmeden bulunan hatalar.
  - Dış hata maliyeti: teslimden sonra bulunan hatalar. Maliyeti daha yüksektir ve şirketin itibarına zarar verir.

**Standartlar ve Kalite Yönetim Sistemi**

- Standartlar kurum standardı, proje standardı veya dünya çapında kabul gören standart olabilir.
- Kurumun bir kalite yönetim sistemi ve kalite el kitabı vardır.
- Her proje kurum standartlarına ek yaparak kendi standart el kitabını oluşturabilir.
- Kalite yönetim sistemi: istenen kalite faktörlerini sağlamak için geliştirici ve kullanıcının birlikte belirlediği yönetim yapısıdır. Sorumlulukları ve kaynakları belirler.

**V Model**

- Sol taraf yazılım geliştirme süreci, sağ taraf yazılım test sürecidir.
- Bileşen testi ile alçak ve yüksek düzey tasarım eşleşmelerinde genellikle doğrulama (verification) yapılır.
- Kullanıcı testinde geçerleme (validation) yapılır. Ürünün gereksinimlere uygun olup olmadığına bakılır.

**Gözden Geçirme ve İnceleme**

- Gözden geçirme: her geliştirme basamağı tamamlanınca durak noktalarında (milestone) yapılan işlerin incelenmesi.
- İlk hedef hataları bulmak ve giderilmesini istemektir.
- Standartlara uygunluk da kontrol edilir. Doğru cevap vermek tek başına yeterli değildir.
- Tipler: formal olmayan gözden geçirme (informal review), yapısal denetim (walkthrough), inceleme (inspection).
- İnceleme (inspection) kod çalıştırılmadan hata bulmayı hedefler. Statik geçerleme tipidir.

**Test Süreci Adımları**

- Testler yazılımı performans, fonksiyonellik, dayanıklılık ve yapı açısından inceler.
- Birim test: test senaryosu (test case), test programları, stub ve driver.
- Bütünlük testi (integration test):
  - Genel tasarımdaki mimari yapı üzerinde yapılır.
  - Aşağıdan yukarıya (bottom-up): en alt modüllerden başlanır, kümeler (cluster) oluşturularak yukarı çıkılır.
  - Yukarıdan aşağıya (top-down): alt modüller yerine yapay modüller (stub) konur, ayrıntılar aşağı doğru açılır.
- Regresyon testi: test edilmiş bir program parçasında değişiklik veya ekleme yapılınca yeniden test edilmesi.
- Sistem testi: yazılımın birlikte çalıştığı ortamla uyumlu çalıştığının onaylanması.
  - Güvenlik testi: dış müdahale ve bilgi hırsızlığına karşı koruma.
  - Dayanıklılık (stres) testi: fazla yüklemede başarılı çıkabilme.
  - Yetenek (performans) testi: işlem süresi.
  - Sistem testi bir modül içinde de uygulanabilir. Modül doğru sonuç verse de süre ve dayanıklılık açısından test edilir.
- Onaylama testi:
  - Kabul muayenesi: kullanıcı, belgeleri izleyerek sistemin doğru çalıştığını onaylar.
  - Düzen (konfigürasyon) testi: sistem belgeleri, kullanıcı belgeleri, test verisi ve test spesifikasyonu tam mı?
  - İkisinden geçen sistem teslim için onaylanır.

**Test Tipleri**

- Saydam kutu (white box) testi:
  - Ayrıntılı tasarımdaki algoritma kullanılır.
  - Modüldeki bütün bağımsız yollar en az bir kez çalıştırılır.
  - Bütün mantıksal kararlar için doğru ve yanlış durumları denenir.
  - Döngüler artış değerleri ve işleyiş açısından değerlendirilir.
  - İç veri yapıları test edilir.
- Temel yol testi (basis path testing):

  - Akış grafı çizilir.
  - McCabe ölçüsü bulunur.
  - Doğrusal bağımsız yolların temel grubu belirlenir.
  - Her temel yoldan bir kez geçecek test verisi hazırlanır.
  - Çıkan sonuç beklenen sonuçla karşılaştırılır.
- Döngü testi: döngülerin kontrol deyimleri ve artışları kontrol edilir. Sonsuz döngü ve döngüye hiç girmeme durumları denenir.
- Kara kutu (black box) testi:
  - Dışarıdan çalıştıran kişinin bakış açısıyla yapılır.
  - İşlevlerin yerine getirildiğine ve girdilerin kabul edildiğine bakılır.
  - Daha az testle farklı durumları denemek için veri sınıfları oluşturulur veya sınırlara bakılır.
  - Eş değer bölümleme (equivalence partitioning): veriler sınıflara ayrılır ve her sınıftan değerle test edilir.
  - Sınır değer analizi (boundary value analysis): hatalar sınırlarda olur varsayımıyla önce sınırlar denenir.
- Gri kutu (gray box) testi iki yaklaşımın karmasıdır.

**Hata Giderme ve Hata Düzeyleri**

- Debugging: test sonucunda bulunan hatanın nedenini bulup düzeltme süreci.
- Hata nedenleri sırayla gereksinim, tasarım, kod ve diğer aşamalardaki hata ve eksikliklerdir.
- Hata düzeyleri ve etkisi:

| Düzey | Test | Teslim |
|---|---|---|
| Ölümcül | Durdurulur | Durdurulur |
| Kritik | Bazı kalite gruplarına göre durdurulur, bazılarına göre devam eder | Durdurulur |
| Büyük | Devam eder | Hoca açıkça belirtmedi |
| Orta | Devam eder | Bazen teslime gidilir |
| Küçük ve görünüm | Devam eder | Teslimi engellemez |

**Ders sonrası notlar**

- Altyazıda “Maccape” McCabe, “Halçtaht” Halstead, “Macconnell” McConnell, “Maccon’un kalite üçgeni” McCall’un kalite üçgenidir.
- Hoca McCall’un gruplarını “işlevsel, değiştirilebilme, çalıştırılabilme ölçütleri” diye andı. Literatürdeki adlar ürün işletimi (product operation), ürün değişikliği (product revision) ve ürün geçişidir (product transition).
- Altyazıdaki “kaçılan box sayısı” kaçırılan hata (bug) sayısı anlamındadır.
- Function point hesabında girdi, çıktı ve dosya sayılarının yanında sorgu sayısı ve dış arayüz sayısı da sayılır.
- Hoca bütünlük testinde stub kavramını yukarıdan aşağıya yaklaşımla açıkladı. Aşağıdan yukarıya yaklaşımda üst modülün yerini sürücü (driver) alır.
- Hoca büyük hatalarda teslimin durumunu açıkça söylemedi. Orta hatalarla bile teslime yalnız bazen gidildiğini belirtti.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.